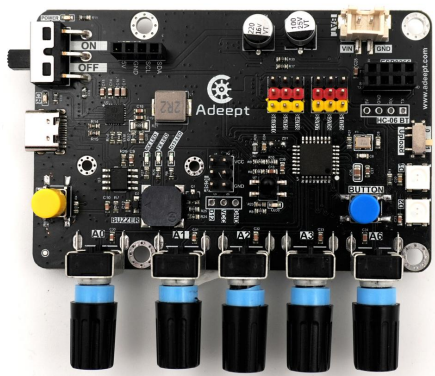


Lesson 10 How to Read the Data of the Potentiometer

10.1 Overview

In this lesson, we will learn how a potentiometer works through an example of rotating it.

10.2 Required Components

Components	Quantity	Picture
Adeept Robotic Arm Control Board	1	
Type-C USB Cable	1	
OLED Screen	1	

10.3 Principle Introduction

As a common electronic component, a potentiometer is essentially a variable resistor. It is widely used in all kinds of electronic equipment, especially in single-chip microcontroller projects based on Arduino, where it is frequently adopted for parameter adjustment.

A potentiometer operates mainly on the principle of resistance voltage division. Its working process is described as follows: When a fixed voltage (such as the 5V commonly used in Arduino systems) is applied across both ends of the resistive element, current flows through it, creating an evenly distributed voltage gradient along the resistor, also known as uniform voltage division. Turning the potentiometer knob or sliding the slider moves the metal wiper across the resistive element and shifts its contact point.

The position of the wiper directly alters the output voltage from the middle pin. The closer the wiper moves to one fixed end pin, the lower the resistance between the wiper and that pin, and the smaller the output voltage. Conversely, the closer the wiper is to the other fixed pin, the higher the resistance between them, resulting in a larger output voltage.

In short, by adjusting the contact position of the metal wiper, the potentiometer taps different voltage levels from the resistive element. It converts mechanical movement such as knob rotation or slider displacement into a corresponding voltage signal, completing the transformation from mechanical quantities to electrical signals.

There are two primary wiring configurations for potentiometers. Among them, the voltage divider mode is the most widely used in Arduino single-chip microcontroller projects, detailed below:

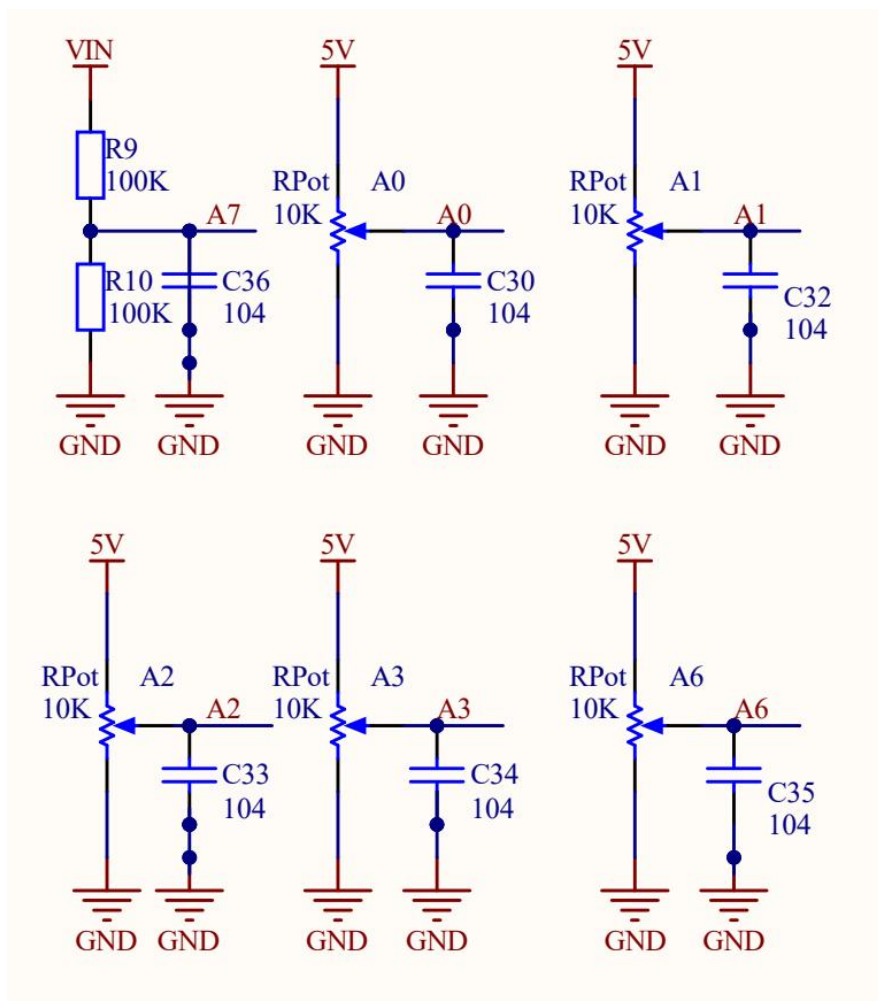
- Voltage Divider Mode (Analog Input, Standard for Arduino)

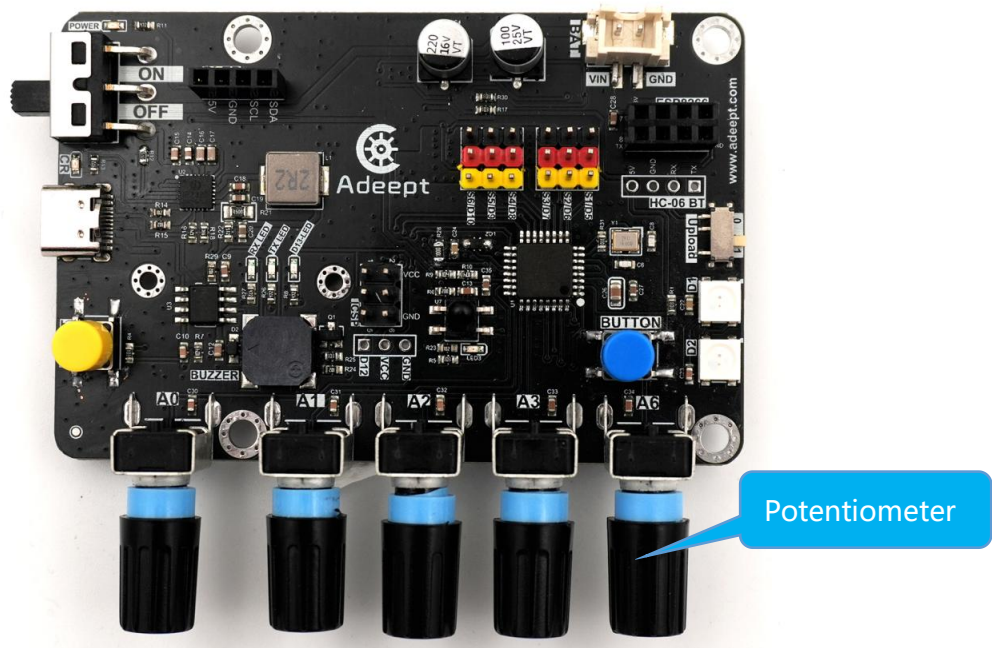
Connect the two fixed end pins of the potentiometer to power (e.g., 5V) and ground (GND) respectively, and link the middle wiper pin to an analog input pin of the microcontroller (A0~A7 on Arduino). The microcontroller then reads the voltage signal output from the middle pin via its built-in ADC (Analog-to-Digital Converter) and converts it into a digital value ranging from 0 to 1023. Rotating the potentiometer knob causes the digital value captured by the analog pin to change continuously between 0 and 1023, enabling adjustment of relevant parameters. In practical projects, this wiring method is used to regulate motor speed, LED brightness, servo rotation angles and more, offering simple operation and smooth adjustment performance.

- Variable Resistor Mode

This wiring method is relatively straightforward. Only the middle wiper pin and one of the fixed end pins are utilized, with the potentiometer acting as an adjustable resistor in the circuit. Turning the knob shifts the wiper position and changes the resistance incorporated into the circuit, which modulates the loop current. This mode suits scenarios requiring basic current regulation, such as coarse adjustment of indicator LED brightness or modifying circuit conduction characteristics where precise control is unnecessary.

10.4 Wiring Diagram





PINS of Adeept Robotic Arm Control Board	Potentiometer
A0	A0
A1	A1
A2	A2
A3	A3
A6	A6

10.5 Demonstration

1. Connect your computer and Adeept Robotic Arm Control Board (Arduino Board) with a USB cable.
2. Open **"10_Potentiometer"** folder in **"/Code"** , double-click **"10_Potentiometer.ino"** .



```

10_Potentiometer | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
10_Potentiometer.ino
1  /*****
2  Website:www.adeept.com
3  E-mail:support@adeept.com
4  *****/
5  #include <Wire.h>
6  #include <SSD1306AsciiWire.h>
7
8  #define OLED_I2C_ADDR  0x3C
9  SSD1306AsciiWire oled;
10 char oledBuf[10];
11 int val1=0;
12 int val2=0;
13 int val3=0;
14 int val4=0;
15 int val5=0;
16 int lastOledVal[5] = {-1, -1, -1, -1, -1};
17
18 void oledInit() {
19   Wire.begin();
20   oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
21   oled.setFont(Adafruit5x7);
22   oled.clear();
23 }

```

3. Select development board and serial port.

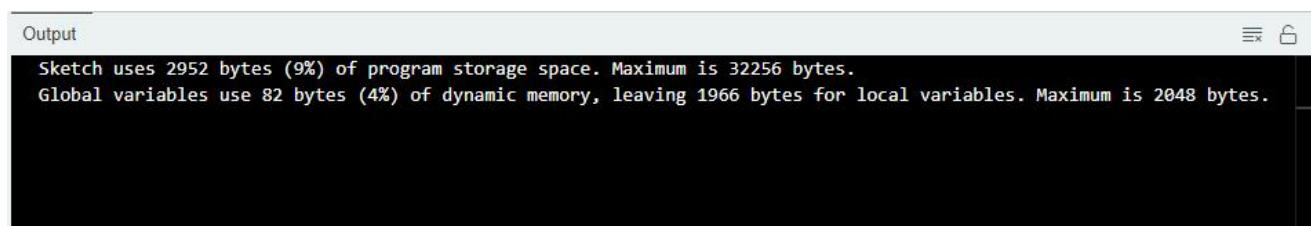
Board: **Tools---**>**Board---**>**Arduino AVR Boards---**>**Arduino Uno**

Port: **Tools --->Port---**>**COMx**

Note: The port number will be different in different computers.



4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.

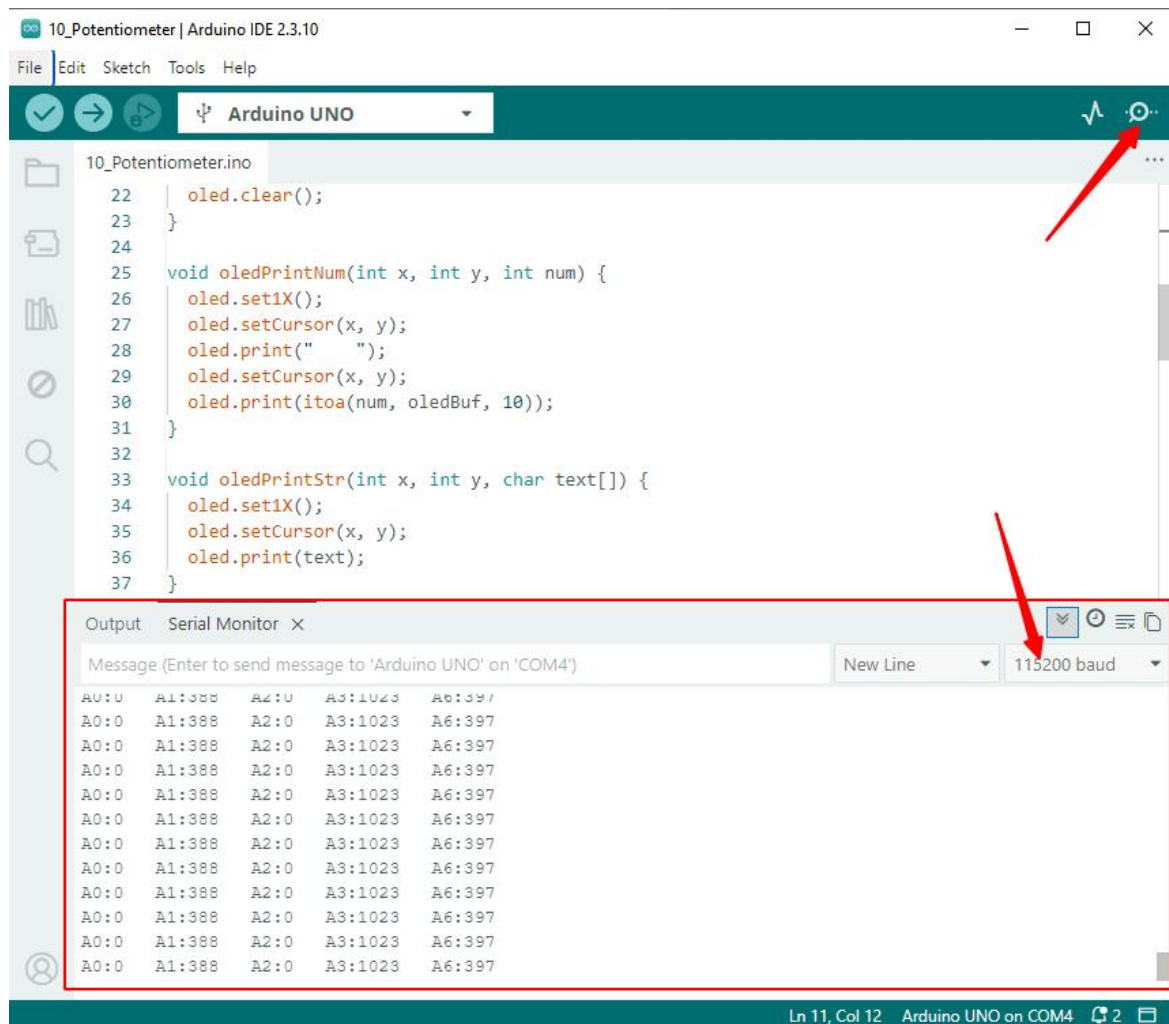


```

Output
Sketch uses 2952 bytes (9%) of program storage space. Maximum is 32256 bytes.
Global variables use 82 bytes (4%) of dynamic memory, leaving 1966 bytes for local variables. Maximum is 2048 bytes.

```

5. After the program runs successfully, you can open the Serial Monitor or check the OLED screen. Rotating the potentiometer will display its current real-time value.



10.6 Code

```

01  /*****
02  Website:www.adeept.com
03  E-mail:support@adeept.com
04  *****/
05  #include <Wire.h>
06  #include <SSD1306AsciiWire.h>
07
08  #define OLED_I2C_ADDR  0x3C
09  SSD1306AsciiWire oled;
10  char oledBuf[10];
11  int val1=0;
12  int val2=0;
13  int val3=0;
14  int val4=0;
15  int val5=0;
16  int lastOledVal[5] = {-1, -1, -1, -1, -1};
17

```

```
18 void oledInit() {
19     Wire.begin();
20     oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
21     oled.setFont(Adafruit5x7);
22     oled.clear();
23 }
24
25 void oledPrintNum(int x, int y, int num) {
26     oled.set1X();
27     oled.setCursor(x, y);
28     oled.print(" ");
29     oled.setCursor(x, y);
30     oled.print(itoa(num, oledBuf, 10));
31 }
32
33 void oledPrintStr(int x, int y, char text[]) {
34     oled.set1X();
35     oled.setCursor(x, y);
36     oled.print(text);
37 }
38
39 void setup() {
40     Serial.begin(115200);
41     oledInit();
42     oled.invertDisplay(true);
43     delay(500);
44     oled.invertDisplay(false);
45     oled.clear();
46 }
47
48 void loop() {
49     val1 = analogRead(0);
50     val2 = analogRead(1);
51     val3 = analogRead(2);
52     val4 = analogRead(3);
53     val5 = analogRead(6);
54
55     Serial.print("A0:");Serial.print(val1);
56     Serial.print("  A1:");Serial.print(val2);
57     Serial.print("  A2:");Serial.print(val3);
58     Serial.print("  A3:");Serial.print(val4);
59     Serial.print("  A6:");Serial.println(val5);
60
61     if(val1 != lastOledVal[0]){
62         oledPrintStr(0, 1, "A0:");
63         oledPrintNum(25, 1, val1);
64         lastOledVal[0] = val1;
65     }
66     if(val2 != lastOledVal[1]){
67         oledPrintStr(64, 1, "A1:");
68         oledPrintNum(88, 1, val2);
69         lastOledVal[1] = val2;
70     }
71     if(val3 != lastOledVal[2]){
72         oledPrintStr(0, 3, "A2:");
73         oledPrintNum(25, 3, val3);
```

```
74     lastOledVal[2] = val3;
75 }
76 if(val4 != lastOledVal[3]){
77     oledPrintStr(64, 3, "A3:");
78     oledPrintNum(88, 3, val4);
79     lastOledVal[3] = val4;
80 }
81 if(val5 != lastOledVal[4]){
82     oledPrintStr(0, 5, "A6:");
83     oledPrintNum(25, 5, val5);
84     lastOledVal[4] = val5;
85 }
86 delay(200);
87 }
```